Containerization Fast Track

Day 3: Kubernetes

United Airlines · Pluralsight instructor-led training

Nick Lotz

Docker concepts and their Kubernetes equivalents

Docker and Compose (Days 1 and 2)	Kubernetes (Day 3)
A container from `docker run`	A Pod with one or more containers
A restart policy	restartPolicy, plus controllers that replace Pods
`HEALTHCHECK` in the Dockerfile	Liveness, readiness and startup probes
Environment variables and env files	ConfigMaps and Secrets
A user-defined network with DNS	A Service and the cluster DNS
A named volume	A PersistentVolumeClaim
A Compose service with several replicas	A Deployment

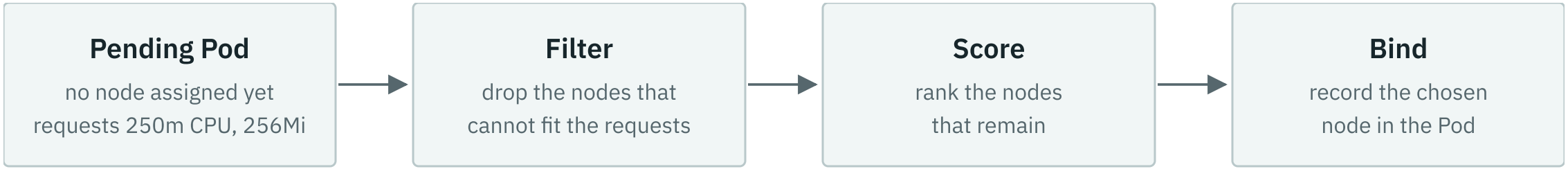
Kubernetes runs the same OCI images that you built and scanned on Days 1 and 2.

What an orchestrator does

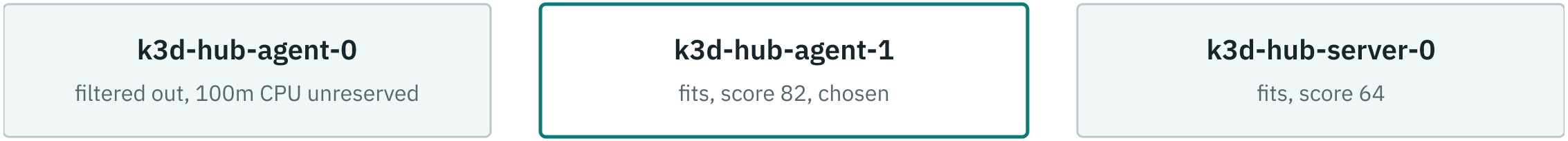
- **Scheduling** picks a node for each container, based on the CPU and memory it requests
- **Self-healing** restarts failed containers and replaces them when a node fails
- **Scaling** runs more or fewer copies of a service
- **Service discovery and load balancing** give a set of copies one stable name and address
- **Rollouts and rollbacks** replace a version step by step and can return to the previous one

Docker Engine runs containers on one host. An orchestrator manages them across a cluster of hosts (Kubernetes Authors, 2026).

What a scheduler does

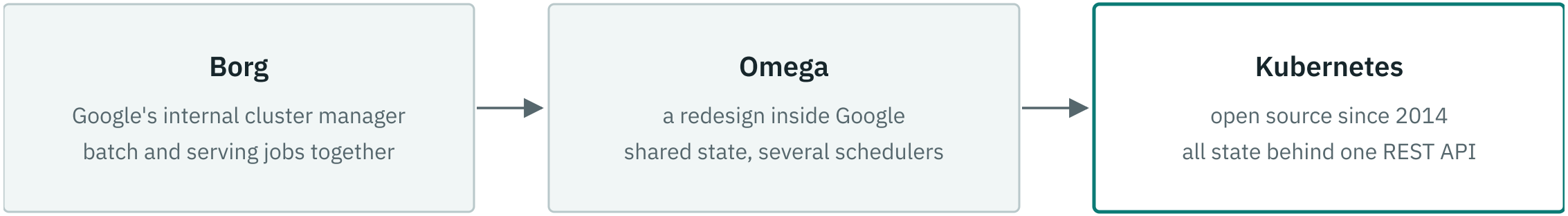


Example with the three nodes of the lab cluster



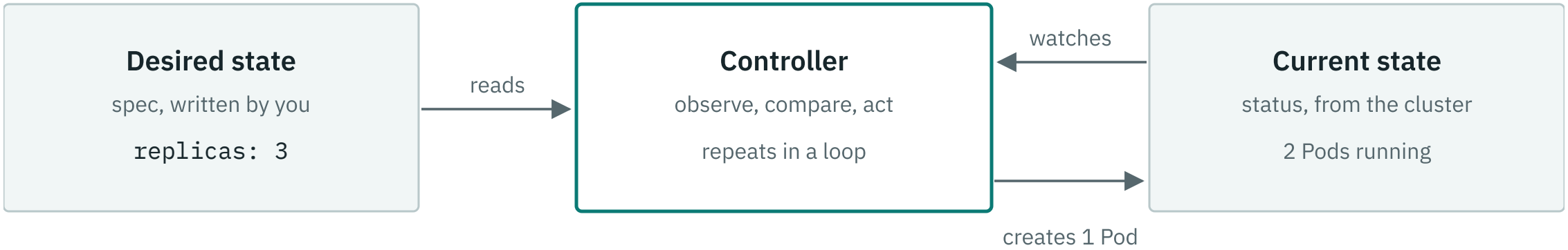
The scheduler compares each Pod's requests with the CPU and memory that are not yet reserved on each node. It ignores current usage (Kubernetes Authors, 2026).

From Borg and Omega to Kubernetes



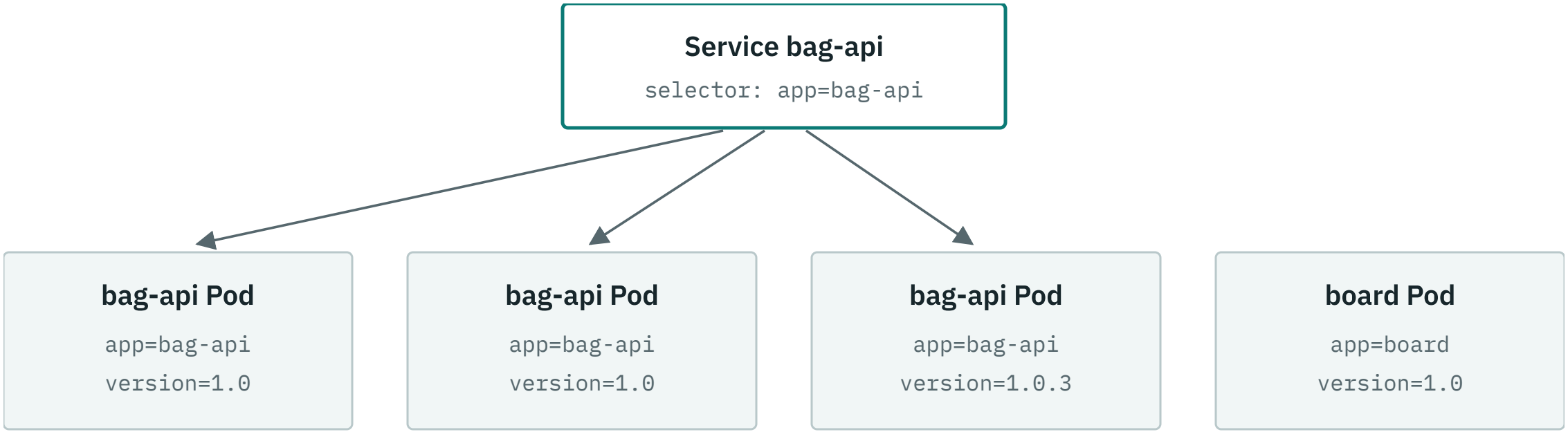
Each system kept the ideas that worked in the one before it (Burns et al., 2016). In 2015, Borg ran hundreds of thousands of jobs on clusters of up to tens of thousands of machines (Verma et al., 2015).

Desired state and reconciliation loops



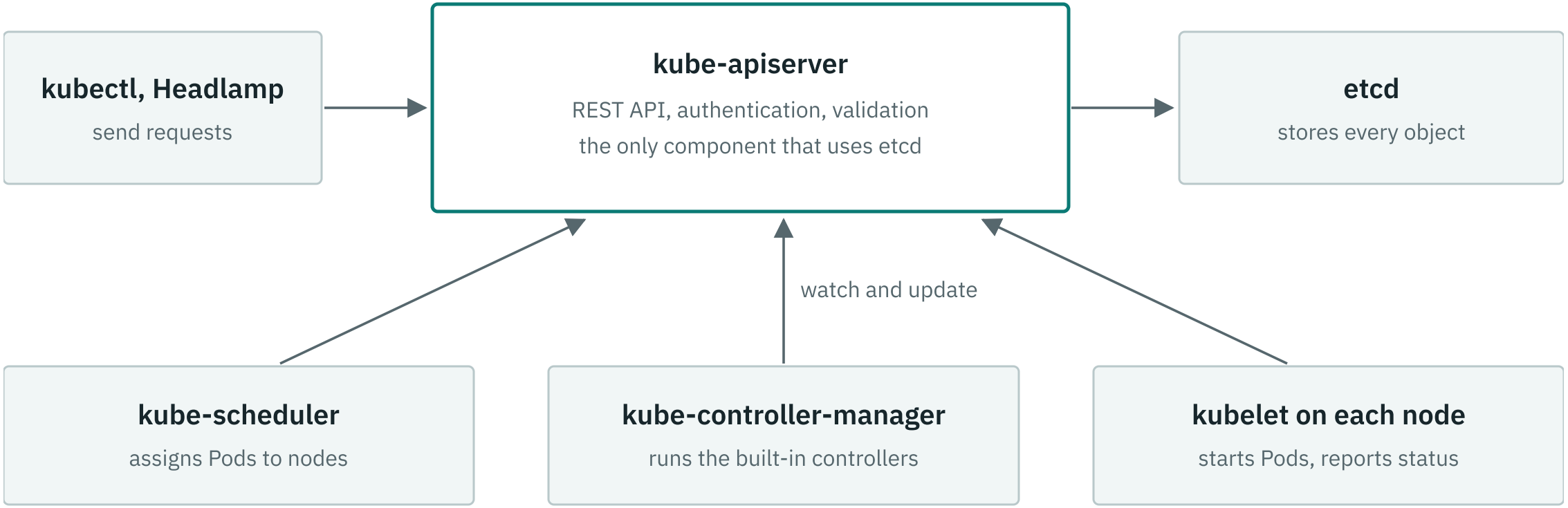
You declare what you want in the spec. Controllers act until the status matches it (Kubernetes Authors, 2026).

Labels and selectors



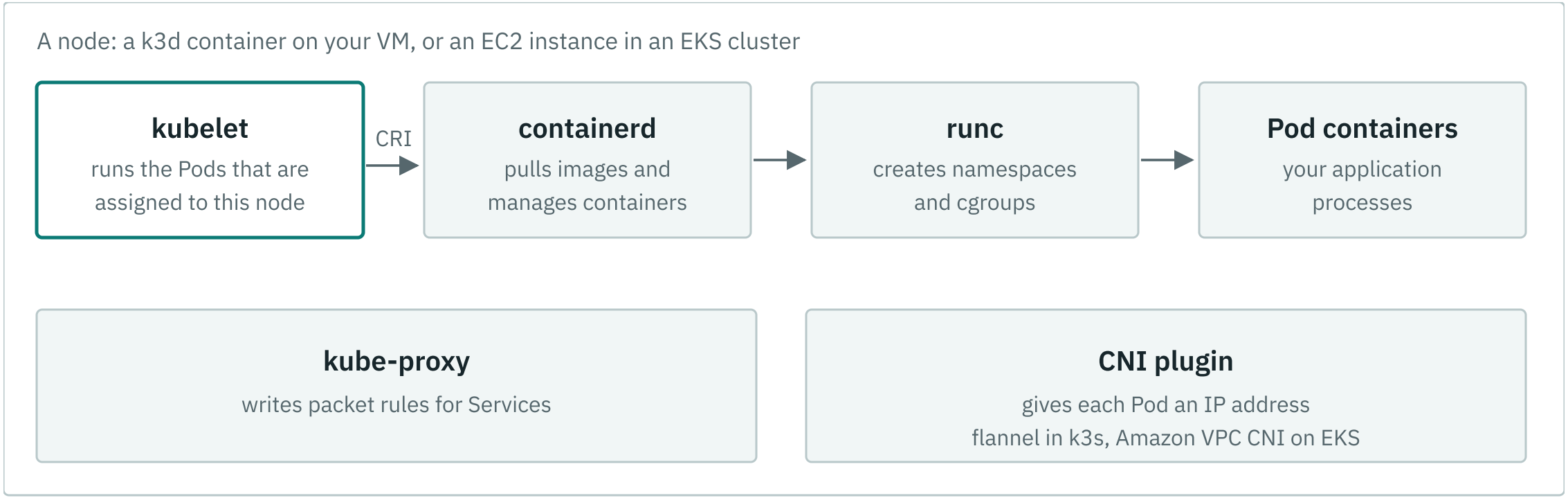
Labels are name=value pairs on any object. Services, Deployments and kubectl find objects by selecting on their labels (Kubernetes Authors, 2026).

The control plane



Every component reads and changes cluster state through the API server (Kubernetes Authors, 2026).

What runs on every node



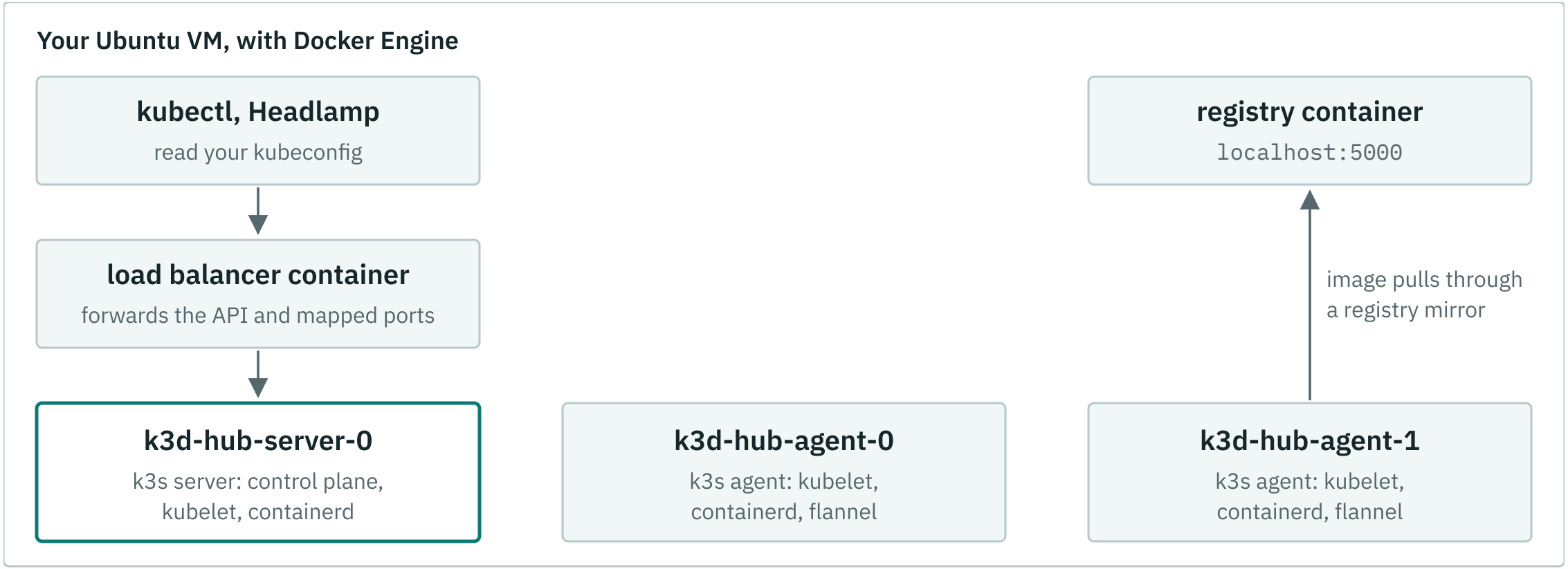
The kubelet talks to the container runtime through the Container Runtime Interface (CRI). Kubernetes removed its built-in Docker support in version 1.24 (Kubernetes Authors, 2026).

Objects and manifests

```
apiVersion: v1
kind: Pod
metadata:
  name: bag-api
  namespace: bagtrack
  labels:
    app: bag-api
spec:
  containers:
  - name: bag-api
    image: localhost:5000/bag-api:1.0
    ports:
    - containerPort: 8000
```

You write `apiVersion`, `kind`, `metadata` and `spec`. The cluster adds `status`, which `kubectl get pod bag-api -n bagtrack -o yaml` shows (Kubernetes Authors, 2026).

How k3d runs a cluster on your lab machine



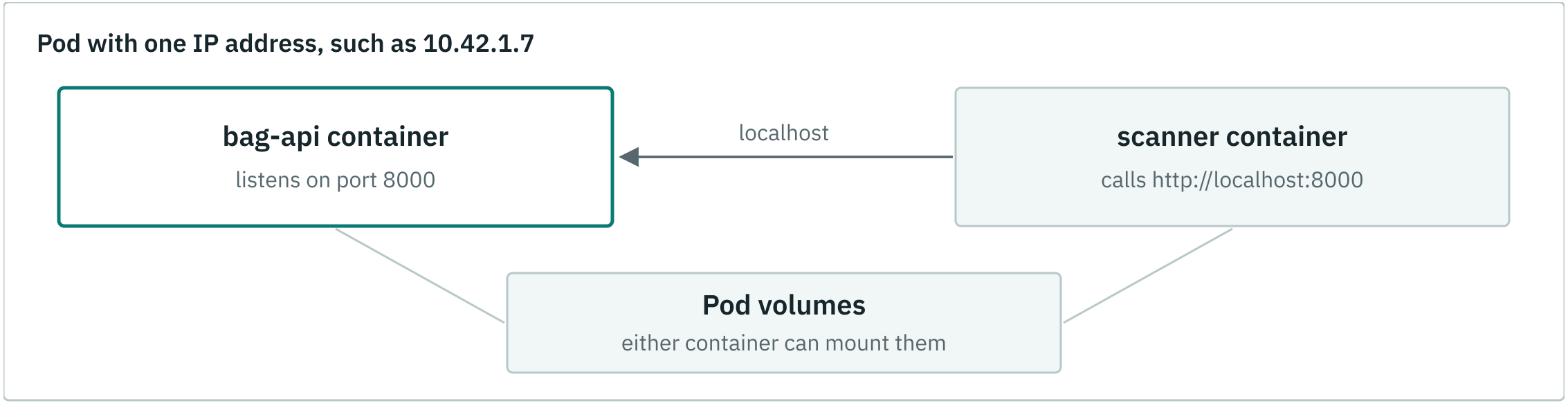
k3d runs each k3s node as a Docker container (k3d Authors, 2026). Pods run inside those containers, under each node's own containerd.

k3d on your VM compared with Amazon EKS

	k3d on your VM	Amazon EKS
Control plane	A k3s server container that you run	Run by AWS across several Availability Zones
Cluster data	SQLite through kine, because there is one server	etcd, encrypted with AWS KMS
Pod network	flannel	Amazon VPC CNI, with Pod IPs from the VPC
Worker nodes	Two k3s agent containers	Managed node groups of EC2 instances, or Fargate profiles
LoadBalancer Services	ServiceLB, on the nodes' ports	A Network Load Balancer from the AWS Load Balancer Controller
Storage	local-path, on one node's disk	Amazon EBS volumes through the EBS CSI driver

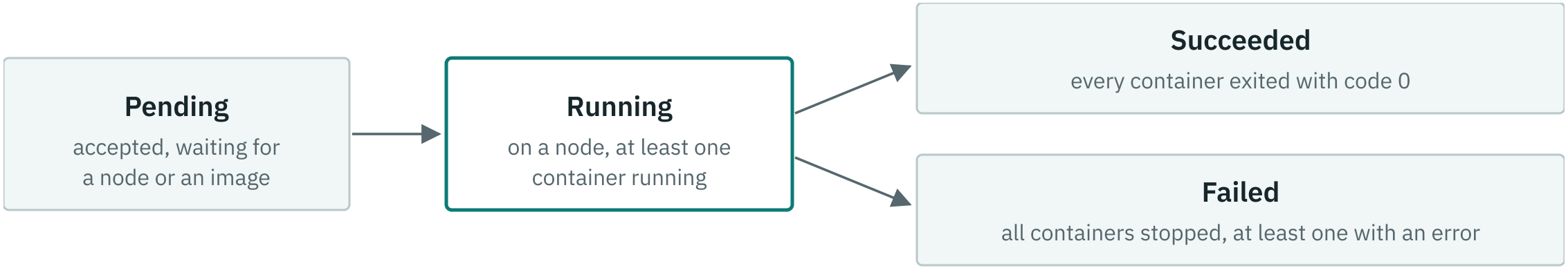
Pods, Deployments and Services use the same manifests on both. Load balancer annotations and StorageClass names differ (Amazon Web Services, 2026; K3s Project Authors, 2026).

The Pod is the unit of scheduling



The scheduler places a whole Pod on one node. Its containers share the Pod's network namespace and can share volumes (Kubernetes Authors, 2026).

The Pod lifecycle and restart policy



restartPolicy, set once for the whole Pod

`Always`	`OnFailure`	`Never`
the default, restart after any exit	restart after a non-zero exit code	leave the container stopped

Before each restart the kubelet waits twice as long as before, from 10 seconds up to 300. While it waits, the container shows CrashLoopBackOff (Kubernetes Authors, 2026).

Liveness, readiness and startup probes

Probe	Question it answers	When it fails	bag-api
Liveness	Is the process still working?	The kubelet restarts the container	`/healthz`
Readiness	Can it serve requests now?	The Pod leaves the Service's endpoints until it passes	`/readyz`, which also checks Postgres
Startup	Has the app finished starting?	The kubelet restarts the container. The other probes wait until it passes.	Not needed

Liveness should check only the process itself. If it also checked Postgres, one database outage would restart every bag-api Pod (Kubernetes Authors, 2026).

Requests and limits

```
containers:
- name: bag-api
  image: localhost:5000/bag-api:1.0
  resources:
    requests:
      cpu: 250m
      memory: 256Mi
    limits:
      cpu: "1"
      memory: 256Mi
```

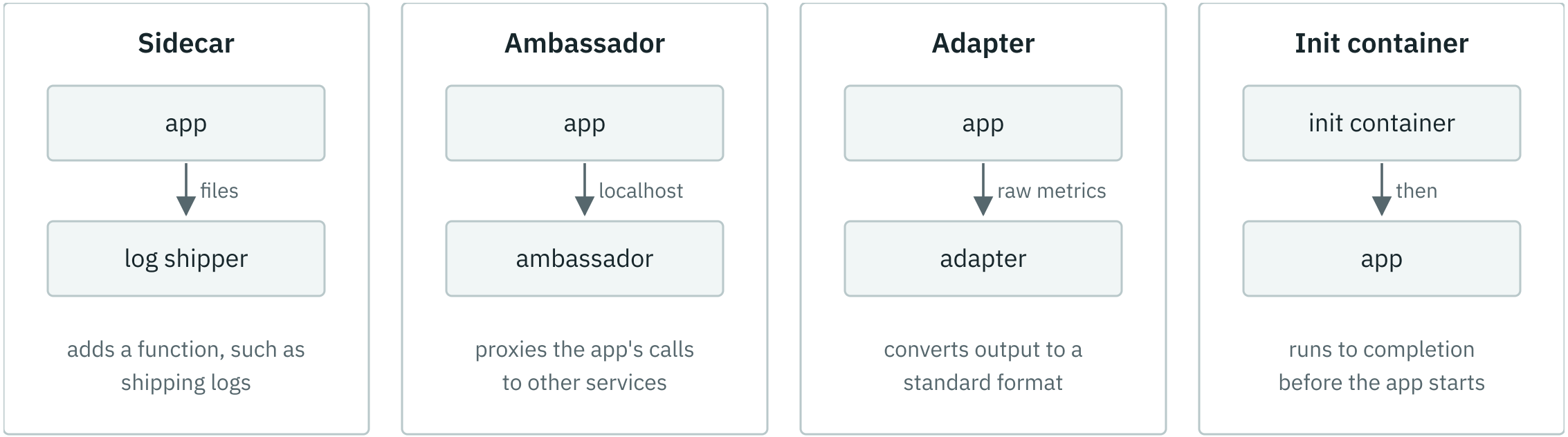
The scheduler uses requests to choose a node. Through cgroups, a CPU limit slows the container down, and a memory limit kills it with exit code 137 and the reason OOMKilled (Kubernetes Authors, 2026).

Quality of service classes

Class	When a Pod gets it	When the node runs short of memory
Guaranteed	Every container sets CPU and memory limits, with requests equal to them	Evicted last
Burstable	At least one container sets a request or a limit	Evicted after all BestEffort Pods
BestEffort	No container sets any request or limit	Evicted first

Kubernetes assigns the class from the resources you set. The bag-api settings on the previous slide give a Burstable Pod (Kubernetes Authors, 2026).

Multi-container patterns



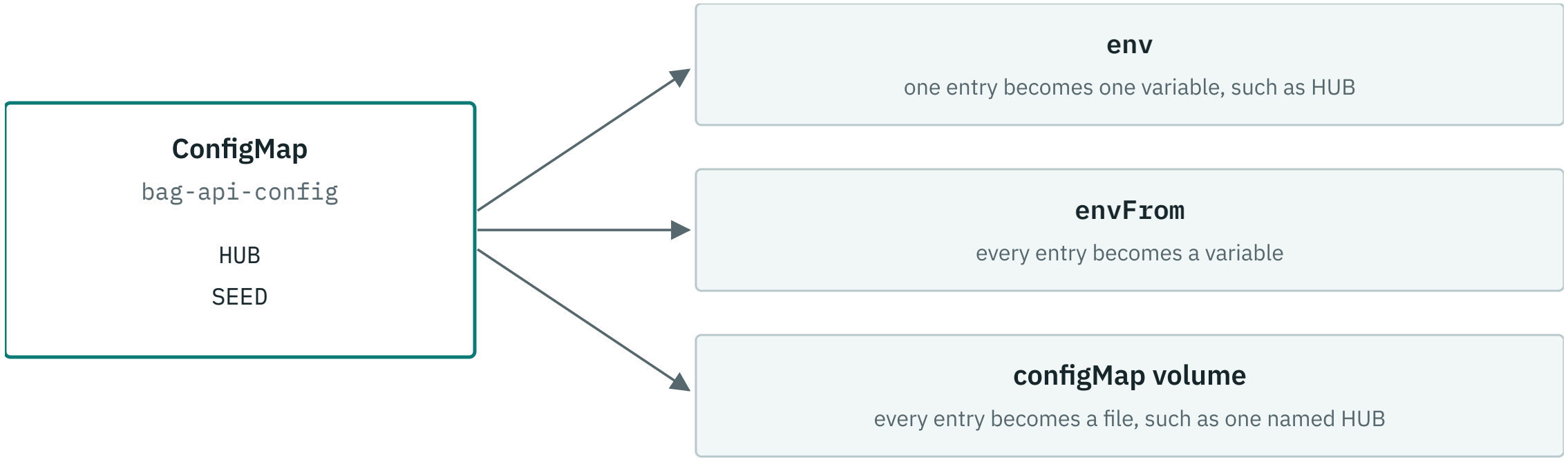
Burns and Oppenheimer (2016) named the sidecar, ambassador and adapter patterns. Init containers are the Kubernetes feature for setup steps that must finish first.

Init containers and native sidecar containers

```
spec:
  initContainers:
  - name: wait-for-db
    image: public.ecr.aws/docker/library/busybox:1.37
    command: ["sh", "-c", "until nc -z bagtrack-db 5432; do sleep 2; done"]
  - name: log-shipper
    image: public.ecr.aws/docker/library/busybox:1.37
    restartPolicy: Always
    command: ["sh", "-c", "tail -F /logs/access.log"]
  containers:
  - name: bag-api
    image: localhost:5000/bag-api:1.0
```

Init containers run one after another and must finish before the app starts. An init container with `restartPolicy: Always` is a native sidecar, stable since Kubernetes 1.33 (Kubernetes Authors, 2026).

Three ways to use a ConfigMap



Variables are set once, when the container starts. Mounted files are updated after the ConfigMap changes (Kubernetes Authors, 2026).

How Kubernetes stores and protects Secrets

- Secret values are only base64-encoded, so anyone who can read a Secret can decode it
- etcd holds Secrets unencrypted unless encryption at rest is turned on (Kubernetes Authors, 2026)
- k3s turns encryption on with `--secrets-encryption`, and EKS encrypts all API data with AWS KMS by default (Amazon Web Services, 2026)
- RBAC limits who can read Secrets, but anyone who can create a Pod in a namespace can mount its Secrets
- External stores such as AWS Secrets Manager keep the value outside the cluster and mount it into Pods

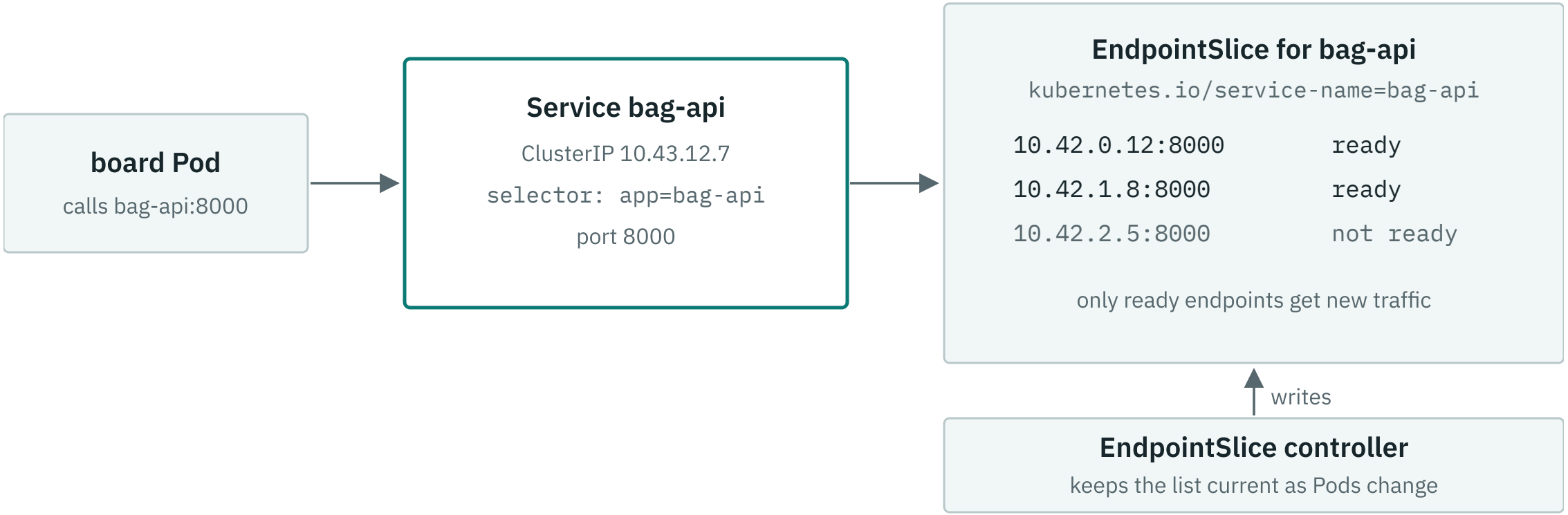
Pods use Secrets in the same ways as ConfigMaps, as variables or as files.

Lab 07: A cluster and its first Pods

`labs/07-pods.md`

- Install kubectl, k3d and Headlamp, and create the three-node cluster `hub`
- Explore the nodes, the system Pods and the API
- Run bag-api as a Pod with liveness and readiness probes
- Give it settings from the ConfigMap `bag-api-config` and the Secret `bag-api-db`
- Run bag-api and the scanner as two containers in one Pod that share localhost

Services and EndpointSlices



A Service gives a changing set of Pods one stable name, IP address and port. The EndpointSlice controller lists the Pods behind it, up to 100 per slice (Kubernetes Authors, 2026).

Service types

Type	Reachable from	What it adds	Typical use
ClusterIP	Inside the cluster	A virtual IP and a DNS name	bag-api, called by the board and the scanner
NodePort	Every node's IP, on a port from 30000 to 32767	A port on every node	Labs and tests
LoadBalancer	Outside, through a load balancer	An external load balancer in front of the nodes or Pods	The board, for users
Headless	Inside the cluster	DNS records for each Pod, with no virtual IP	Postgres in a StatefulSet

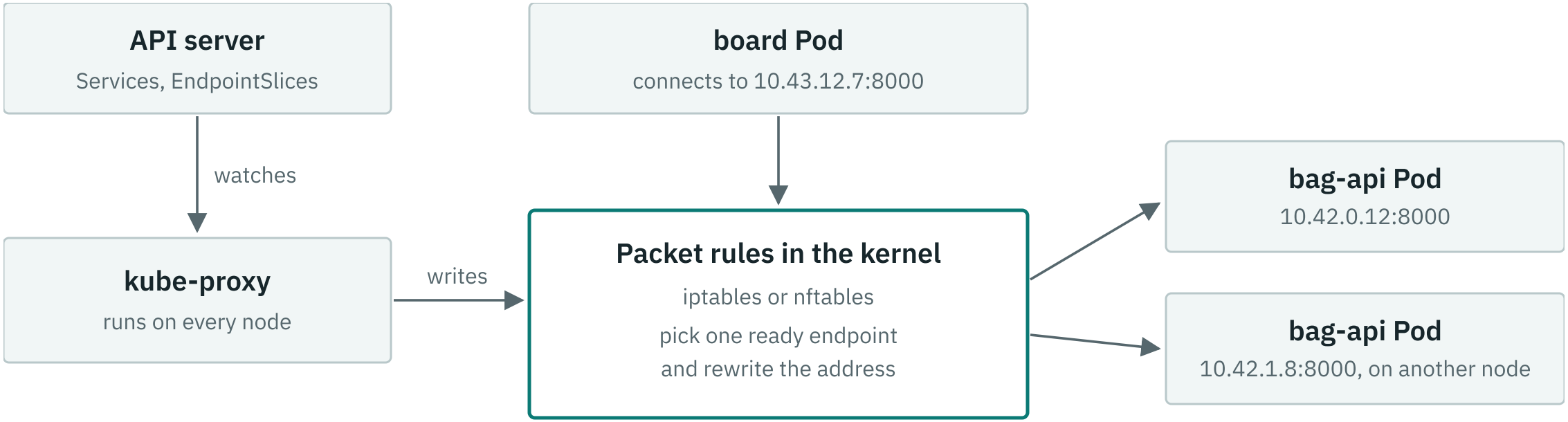
NodePort includes a ClusterIP, and LoadBalancer includes a NodePort by default. A headless Service sets `clusterIP: None` (Kubernetes Authors, 2026).

DNS names for Services

Name that a Pod looks up	Works from
`bag-api`	Pods in the same namespace
`bag-api.bagtrack`	Pods in any namespace
`bag-api.bagtrack.svc.cluster.local`	Anywhere in the cluster, as the fully qualified name
`bagtrack-db-0.bagtrack-db`	One Pod of a StatefulSet, through its headless Service

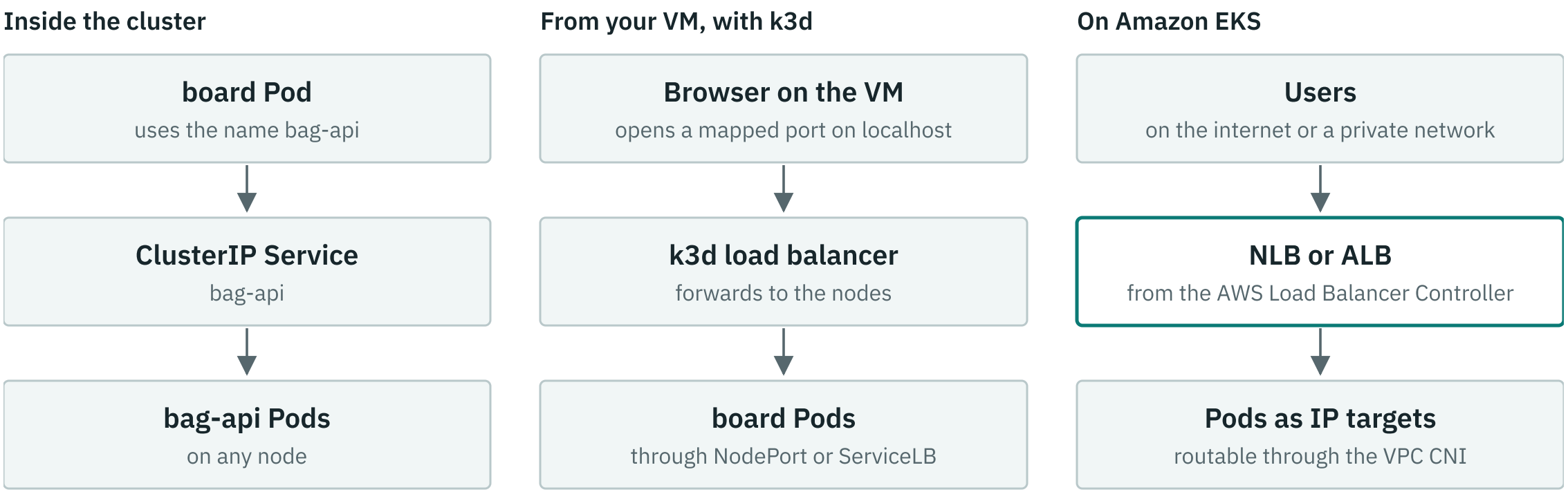
CoreDNS answers these lookups. The kubelet writes search domains into each Pod's `/etc/resolv.conf`, which makes the short names work (Kubernetes Authors, 2026).

What kube-proxy programs



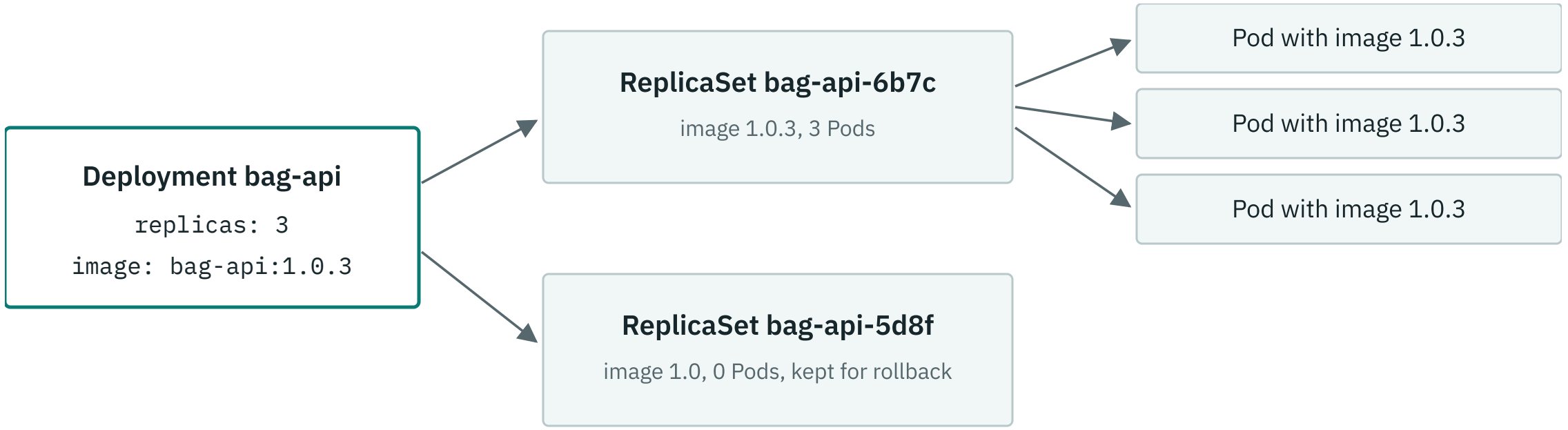
Nothing listens on the ClusterIP. The kernel sends each connection to one ready Pod (Kubernetes Authors, 2026).

Internal and external access, and how EKS provides it



On EKS, the AWS Load Balancer Controller creates a Network Load Balancer for a LoadBalancer Service and an Application Load Balancer for an Ingress (Amazon Web Services, 2026).

Deployments, ReplicaSets and Pods



You change the Deployment. It creates one ReplicaSet for each version of the Pod template, and each ReplicaSet keeps its number of Pods running (Kubernetes Authors, 2026).

Scaling a Deployment

```
# Run five copies of bag-api instead of three
kubectl -n bagtrack scale deployment/bag-api --replicas=5

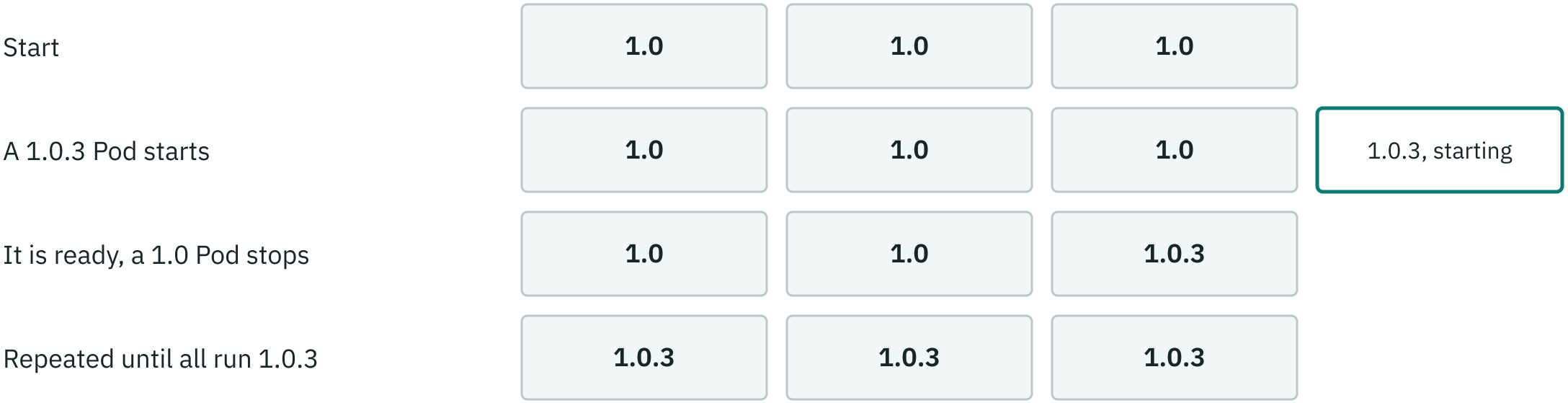
# See which node each Pod runs on
kubectl -n bagtrack get pods -l app=bag-api -o wide

# Delete one Pod and watch its replacement start
kubectl -n bagtrack delete pod <name-of-a-bag-api-pod>
kubectl -n bagtrack get pods -l app=bag-api --watch
```

Scaling changes one number in the spec, and the ReplicaSet controller creates or deletes Pods to match. A HorizontalPodAutoscaler changes it from measured CPU or memory use (Kubernetes Authors, 2026).

Rolling update parameters

Example from image 1.0 to 1.0.3, with replicas 3, maxSurge 1 and maxUnavailable 0



maxSurge sets how many extra Pods may exist during the update, and maxUnavailable sets how many may be missing. Both default to 25% of the replicas (Kubernetes Authors, 2026).

Rollout history and rollback

```
kubectl -n bagtrack set image deployment/bag-api bag-api=localhost:5000/bag-api:1.0.3
kubectl -n bagtrack rollout status deployment/bag-api

kubectl -n bagtrack rollout history deployment/bag-api
kubectl -n bagtrack rollout undo deployment/bag-api
kubectl -n bagtrack rollout undo deployment/bag-api --to-revision=1
```

A rollout that stops making progress is reported as ProgressDeadlineExceeded, after 600 seconds by default. Kubernetes takes no other action, so you or your deployment tool start the rollback (Kubernetes Authors, 2026).

Lab 08: Deployments and Services

`labs/08-deployments-services.md`

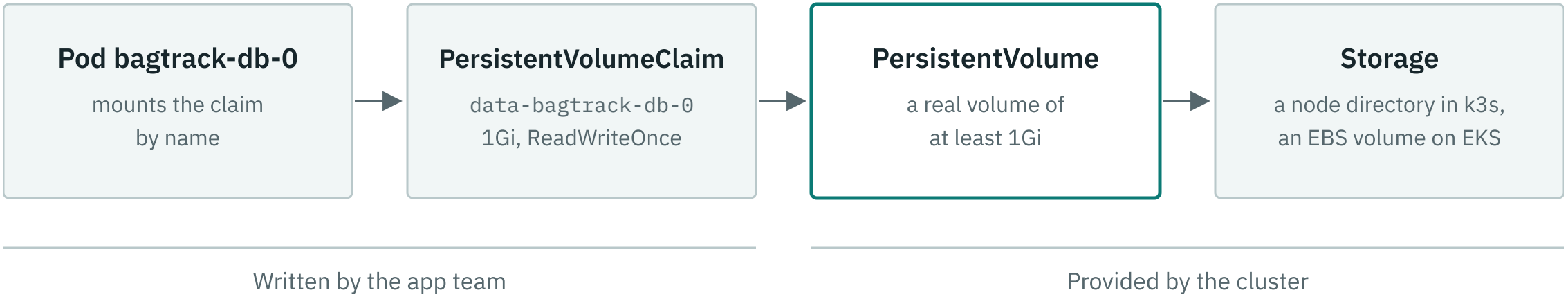
- Deploy bag-api with three replicas, plus the board and the scanner
- Reach bag-api through a ClusterIP Service and its DNS name from a debug Pod
- Open the board in the browser through a LoadBalancer Service and a NodePort
- Refresh the board to see each replica's own bags, and delete Pods to see self-healing
- Roll out the pipeline's tag, such as `1.0.3`, break a rollout with `9.9`, and undo it

Volumes in a Pod

Volume type	Where the data is	How long it lasts	Typical use
`emptyDir`	The node's disk, or memory	As long as the Pod	Scratch files, and files shared between containers
`configMap`, `secret`	Objects in the API	As long as the object	Configuration files
`hostPath`	A path on the node	As long as the node	Node agents only
`persistentVolumeClaim`	Storage outside the Pod	Independent of any Pod	Databases

hostPath exposes the node's filesystem to the Pod. The Baseline Pod Security Standard forbids it (Kubernetes Authors, 2026).

PersistentVolumes and PersistentVolumeClaims



Kubernetes binds each claim to a matching volume. The volume and its data outlive the Pods that use it (Kubernetes Authors, 2026).

StorageClasses and dynamic provisioning

	k3s on your VM	Amazon EKS
StorageClass	`local-path`, the default	One that you define, such as `gp3`
Provisioner	Local Path Provisioner	Amazon EBS CSI driver
Where the data is	A directory under `/var/lib/rancher/k3s/storage` on one node	An EBS volume in one Availability Zone
Effect on scheduling	The Pod stays on that node	The Pod stays in that zone

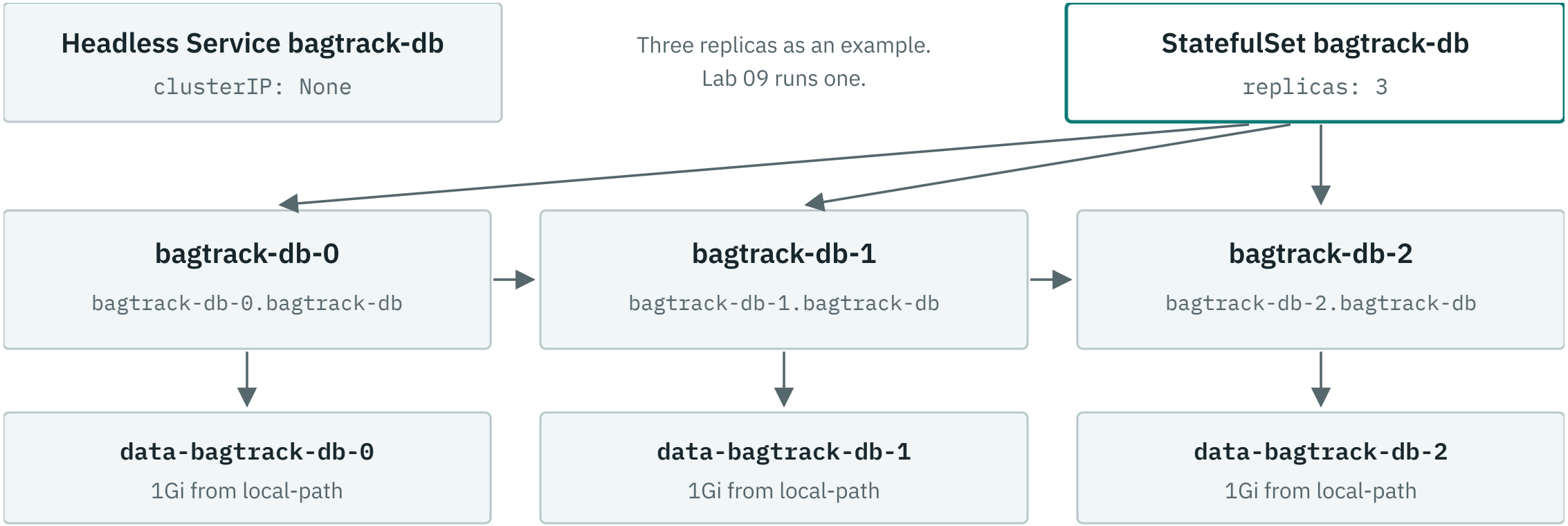
A claim names a StorageClass. The class's provisioner creates a matching PersistentVolume on demand (K3s Project Authors, 2026; Amazon Web Services, 2026).

Access modes and reclaim policy

Setting	Value	Meaning
Access mode	`ReadWriteOnce`	Read-write from one node at a time
	`ReadWriteOncePod`	Read-write from one Pod
	`ReadOnlyMany`	Read-only from many nodes
	`ReadWriteMany`	Read-write from many nodes, for example Amazon EFS
Reclaim policy	`Delete`	Deleting the claim deletes the volume and its data
	`Retain`	The volume and its data stay until an administrator removes them

Dynamically provisioned volumes take the reclaim policy of their StorageClass, which is Delete by default (Kubernetes Authors, 2026).

StatefulSets



Each Pod keeps its name, DNS name and claim when it is replaced. Pods start one at a time in order and stop in reverse order (Kubernetes Authors, 2026).

How a Job runs Pods to completion

Field	Meaning	Default
`completions`	How many Pods must succeed	1
`parallelism`	How many Pods may run at once	1
`backoffLimit`	Failures allowed before the Job fails	6
`activeDeadlineSeconds`	Time limit for the whole Job	None
`ttlSecondsAfterFinished`	Deletes the finished Job after this many seconds	None

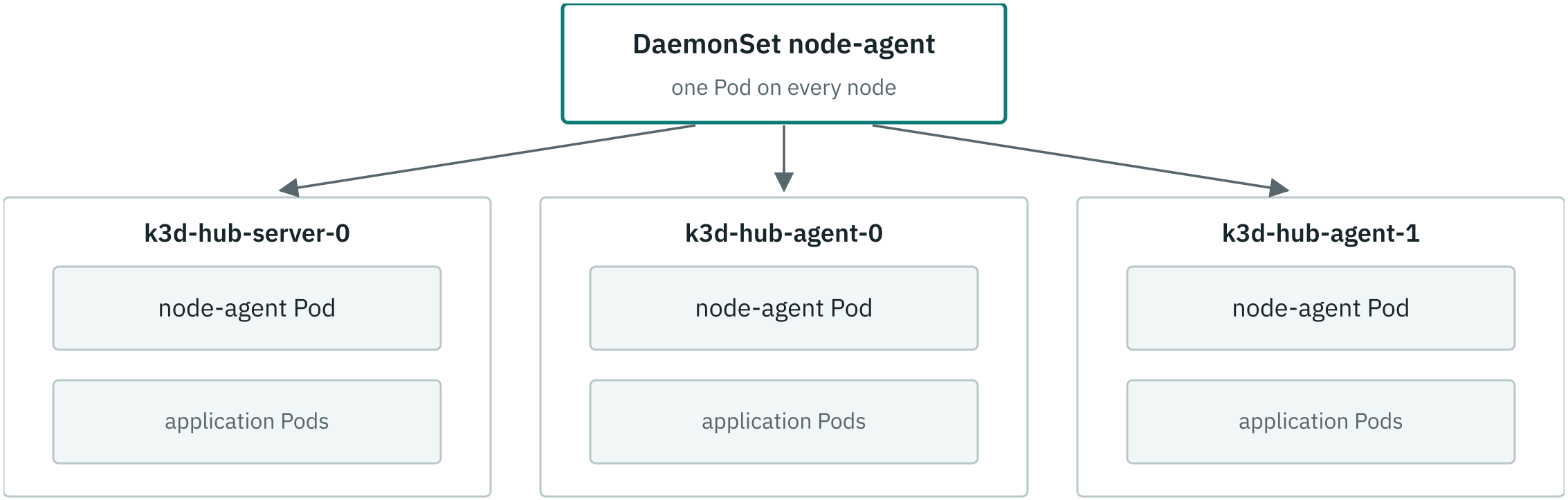
A Job's Pods use restartPolicy OnFailure or Never. The Job succeeds when enough Pods exit with code 0 (Kubernetes Authors, 2026).

CronJobs

```
apiVersion: batch/v1
kind: CronJob
metadata: {name: bag-report, namespace: bagtrack}
spec:
  schedule: "* * * * *"
  concurrencyPolicy: Forbid    # optional, not set in the lab
  jobTemplate:
    spec:
      template:
        spec:
          restartPolicy: OnFailure
          containers:
          - name: report
            image: localhost:5000/scanner:1.0
            args: ["report"]
```

The CronJob controller creates a Job at each scheduled time. concurrencyPolicy decides what happens when the previous Job is still running (Kubernetes Authors, 2026).

DaemonSets



DaemonSets run node-level agents such as log collectors and network plugins. A node that joins the cluster gets the Pod automatically (Kubernetes Authors, 2026).

Lab 09: Storage, StatefulSets and batch work

`labs/09-state-and-batch.md`

- Run Postgres as the StatefulSet `bagtrack-db` with a headless Service and a 1Gi local-path claim
- Delete `bagtrack-db-0` and check that the data survives
- Make bag-api wait for the database with an init container, and load data with a seed Job
- Run `scanner report` as a CronJob every minute, and a node-agent DaemonSet
- Share files between a writer and a reader container through an emptyDir

Scaling approaches compared

Need	Docker and Compose	Amazon ECS	Kubernetes
Run more copies	`--scale` or `deploy.replicas`, on one host	The service's desired count	The Deployment's replicas
Scale automatically	Not built in	Service Auto Scaling	HorizontalPodAutoscaler
Spread the load	DNS round robin on the network	An ALB target group	A Service and kube-proxy
Add capacity	A bigger host	Fargate, or more EC2 instances	More nodes, or Fargate on EKS

Every platform scales a stateless service by adding copies behind one address. Stateful parts such as Postgres need a different plan (Amazon Web Services, 2026; Kubernetes Authors, 2026).

Resilience approaches compared

Need	Docker and Compose	Amazon ECS	Kubernetes
Restart a failed container	A restart policy	The service replaces the task	The kubelet restarts it, and the ReplicaSet replaces lost Pods
Detect an unhealthy app	HEALTHCHECK and `depends_on` conditions	Container and target group health checks	Liveness, readiness and startup probes
Survive a host or zone failure	Not possible on one host	Tasks spread across Availability Zones	New Pods on other nodes and zones
Undo a bad release	By hand	Deployment circuit breaker with rollback	`kubectl rollout undo`, or a CD tool

ECS and Kubernetes replace failed copies on other hosts. Docker and Compose cannot, because each manages one host (Amazon Web Services, 2026; Kubernetes Authors, 2026).

What to learn next

- Helm or Kustomize, to package manifests and vary them between environments
- Ingress and the Gateway API, for HTTP routing into a cluster
- Node autoscaling with Cluster Autoscaler or Karpenter, next to the HorizontalPodAutoscaler
- Metrics, logs and traces with Prometheus and OpenTelemetry
- Deployments in the Harness platform, with services, environments, approvals and rollback for ECS and Kubernetes (Harness, 2026)

Each topic builds on the Pods, Services and Deployments from today.

References

Amazon Web Services. (2026). *Amazon EKS User Guide* and *Amazon ECS Developer Guide*. docs.aws.amazon.com

Burns, B., Grant, B., Oppenheimer, D., Brewer, E., and Wilkes, J. (2016). Borg, Omega, and Kubernetes. *ACM Queue*, 14(1).

Burns, B., and Oppenheimer, D. (2016). Design patterns for container-based distributed systems. *HotCloud '16*, USENIX.

Harness. (2026). *Harness CD documentation*. developer.harness.io

k3d Authors. (2026). *k3d documentation*. k3d.io

K3s Project Authors. (2026). *K3s documentation*. docs.k3s.io

Kubernetes Authors. (2026). *Kubernetes documentation*: cluster architecture, controllers, scheduling, Pods, probes, resource management, ConfigMaps, Secrets, Services, DNS, kube-proxy, Deployments, volumes, persistent volumes, StatefulSets, Jobs, CronJobs, DaemonSets, Pod Security Standards. kubernetes.io/docs

Verma, A., Pedrosa, L., Korupolu, M., Oppenheimer, D., Tune, E., and Wilkes, J. (2015). Large-scale cluster management at Google with Borg. *EuroSys 2015*.

Wiggins, A. (2011). *The Twelve-Factor App*. 12factor.net